

Project Proposal: AutoDoc AI

1. Introduction

Background

In most software projects, documentation is often an afterthought. Developers are usually preoccupied with writing code, fixing issues, and deploying features—leaving little room for properly documenting the system. Over time, this leads to incomplete or outdated documentation, which makes onboarding, collaboration, and maintenance more difficult.

Objective

This project proposes an AI-driven solution that automates the generation of core documentation artifacts for GitHub repositories. The goal is to minimize manual effort while improving the quality and consistency of documentation. The tool will generate structured README files and interactive Codelab tutorials based on the codebase of the repository.

Solution Summary

AutoDoc AI is a web-based application built with Streamlit that allows users to input a GitHub repository URL and receive an automatically generated:

- README.md file summarizing project purpose, usage, and structure
- Interactive Codelab tutorial created using the Claat framework
- GitHub integration to commit or push documentation files directly

All content is generated using Large Language Models (LLMs) and autonomous agents, offering a complete documentation solution with minimal developer input.

2. Project Overview

Scope

Data Sources

- **Unstructured:** GitHub repositories containing python (.py) and SQL (.sql) code files
- **Structured:** Project-specific metadata stored in databases (Snowflake)

Technologies Used

- **Frontend:** Streamlit
- **Backend:** FastAPI
- **Agent Framework:** LangGraph
- **Language Model:** GPT-4o mini
- **Database:** Snowflake
- **Context Retrieval / GitHub CI-CD:** Model Context Protocol Servers
- **Codelab Generation:** Claat framework

Expected Outputs

- Auto-generated README.md files
- Interactive tutorial links based on Codelab
- Cloud-hosted deployment of the application
- CI/CD-enabled GitHub repo

Stakeholders

The primary end users of this project are developers who want to automatically generate high-quality documentation for their GitHub project repositories. These developers may have limited time or resources to manually write comprehensive documentation, and this tool streamlines the process by extracting code, analyzing it with AI agents, and generating a structured README.md file. This helps them maintain up-to-date, consistent documentation, improve project readability, and enhance collaboration and onboarding for new contributors.

3. Problem Statement

Key Challenges

- **Time-Intensive Documentation:** Developers find it difficult to allocate time to write detailed documentation for each stage of the development lifecycle.
- **Limited Codebase Understanding:** Contributors often face delays due to lack of context or architectural clarity when exploring unfamiliar repositories.
- **Out-of-Sync Documentation:** Documentation quickly becomes outdated when not actively maintained in parallel with the code.

Opportunities

- **Efficiency and Time-Saving:** The tool significantly reduces the time developers spend on documentation by automating most of the process.
 - **Version-Safe Integration:** Automatic push or pull request generation ensures that documentation is always synchronized with the repository.
 - **Improved Project Adoption:** Projects with clear and professional documentation are easier to understand and adopt, encouraging external contributions and use.
 - **Reusable and Scalable:** AutoDoc AI can be deployed across multiple repositories, making it valuable for organizations looking to enforce standard documentation practices.
-

4. Implementation Methodology

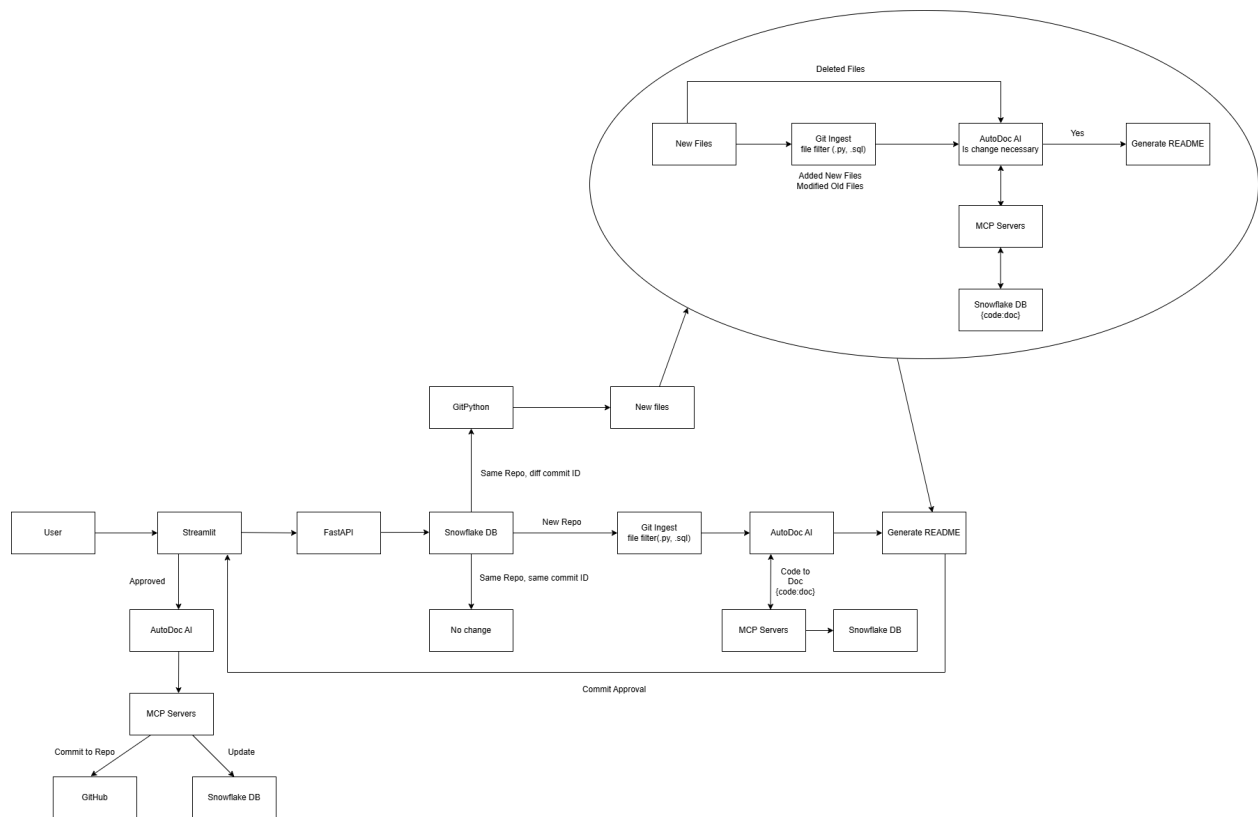
Data Sources

- **Unstructured:** GitHub repositories containing python (.py) and SQL (.sql) code files
- **Structured:** Project-specific metadata stored in databases (Snowflake)

Technologies and Tools

- **Frontend:** Streamlit
- **Backend:** FastAPI
- **Data Transformation:** GitIngest
- **Agent Framework:** LangGraph
- **Language Model:** GPT-4o mini
- **Database:** Snowflake
- **Context Retrieval / GitHub CI-CD:** Model Context Protocol Servers
- **Codelab Generation:** Claat

Data Pipeline Design



Workflow Pipeline

1. **Frontend UI:** User inputs the git repository whose documentations should be generated

2. **Backend:** FastAPI is used to integrate the streamlit to backend architecture
 3. **Historical check:** Check if the repository exists in the Snowflake Database along with its commit id.
 - a. **Present with same commit ID:** No change
 - b. **Not present:** Data Extraction -> AutoDoc AI (Code to Words) -> MCP Server -> Snowflake DB -> AutoDoc AI (Generate Document) -> User Approval -> MCP Server -> Commit Document -> Update Snowflake DB
 - c. **Present with different commit ID:** Data Extraction -> AutoDoc AI (Code to Words) -> MCP Server -> Snowflake DB -> AutoDoc AI (Is change needed) -> AutoDoc AI (Generate Document) -> User Approval -> MCP Server -> Commit Document -> Update Snowflake DB
 4. **Data Extraction:** Use GitIngest to gather repository data from GitHub and transform code files into structured text.
 5. **AutoDoc AI (Code to Words):** Implement an AI Agent to convert the code files into words for step by step documentation.
 6. **MCP Servers (Context Retrieval):** Use MCP Servers to get the context from Snowflake.
 7. **Snowflake DB:** Store the documented files along with the code files in Snowflake DB and fetch the data for context to create the final README.md file.
 8. **AutoDoc AI (Is change needed):** Provide context (code files, its documented files to AI Agent and ask it to make a decision on if a change in document is necessary
 9. **AutoDoc AI (Generate Document):** Generate a README.md file for the given GitHub repository.
 10. **User Approval:** Show the generated README.md file to the user via Streamlit and take approval to commit it to the GitHub repository.
 11. **MCP Servers (Commit Document):** Automate and commit using MCP Servers and update the Snowflake DB.
-

5. Project Plan and Timeline

Phase	Task	Deliverables	Timeline
-------	------	--------------	----------

Phase 1	Data processing pipeline	Data pipeline logs	April 7–8
Phase 2	Agent implementation and documentation generation	README, Codelab	April 9–12
Phase 3	GitHub integration and cloud deployment	Fully documented repo with live Codelab	April 13–18

6. Resources and Team

- **Asavari:** Model Context Protocol (MCP), Backend (FastAPI), and frontend (Streamlit)
- **Hishita:** GitHub automation for commits and pull requests
- **Pratik:** Data pipeline development and agentic AI workflow integration

7. Risks and Mitigation Strategies

Risk	Issue	Mitigation
Incorrect AI Output	LLMs may misunderstand complex logic	Incorporate validation layers and human review checkpoints
LLM Constraints	Some codebases may be too large or ambiguous	Use modular prompts, file-wise summarization, and chunking
Privacy Concerns	Handling of private repos requires security measures	Use encrypted access tokens and secure storage protocols

8. Expected Outcomes and Benefits

- A GitHub repository containing detailed, AI-generated documentation
 - A shareable, interactive Codelab tutorial
 - CI/CD-enabled updates through MCP Servers for GitHub
 - A deployed solution that can be reused for other repositories
-

9. Conclusion

AutoDoc AI streamlines documentation by applying artificial intelligence to one of the most time-consuming aspects of development. It allows developers to:

- Automatically produce consistent and informative README files
 - Interactive Codelab tutorials without manual effort
 - Commit documentation directly to GitHub with minimal setup
-